

HDOC Day-28

1) Write a C program to:

- a) Read the number of rows and columns from the user. Ensure that both values are positive. If not, repeat to get the desired input(s).
- b) Declare a variable-length 2D integer array using the given dimensions.
- c) Read the array elements column by column using nested loops, with the outer loop iterating over columns and the inner loop over rows.
- d) Print the elements in row-major order using a single loop and a linear index.

Solution - C Program

```
#include <stdio.h>

int main()
{
    int rows, cols;

    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        printf("Enter number of columns: ");
        scanf("%d", &cols);

        if (rows <= 0 || cols <= 0)
            printf("Rows and columns must be positive. Enter again.\n");
    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];

    printf("Enter array elements column by column:\n");
    for (int j = 0; j < cols; j++)
    {
        for (int i = 0; i < rows; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    printf("Array elements in row-major order:\n");
    for (int k = 0; k < rows * cols; k++)
    {
        int i = k / cols;
        int j = k % cols;
        printf("%d ", a[i][j]);
        if ((k + 1) % cols == 0)
            printf("\n");
    }
}
```

```

    }

    return 0;
}

```

2) Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays.

Solution - C Program

```

#include <stdio.h>

int main()
{
    int rows, cols;

    do
    {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);
    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];
    int b[rows * cols];

    printf("Enter elements of the 2D array:\n");
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", &a[i][j]);

    int k = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            b[k++] = a[i][j];

    printf("2D array:\n");
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    printf("1D array in row-major order:\n");
    for (int i = 0; i < rows * cols; i++)
        printf("%d ", b[i]);

    printf("\n");
    return 0;
}

```

Result

Both programs were implemented successfully using variable-length arrays. The required array input, row-major traversal, copying, and display operations were performed correctly.

HDOC Day-28 - Test Case Screenshots

Question 1 - Compilation

```
(kali@kali)-[~/Desktop/apple]
$ gcc hdoc28.c -o hdoc28

(kali@kali)-[~/Desktop/apple]
$
```

Question 1 - Test Case 1 (2 x 3)

```
(kali@kali)-[~/Desktop/apple]
$ ./hdoc28
Enter number of rows: 2
Enter number of columns: 3
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4
a[0][2] = 5
a[1][2] = 6
Array elements in row-major order:
1 3 5
2 4 6
```

Question 1 - Test Case 2 (Invalid dimensions, then 3 x 2)

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc28
Enter number of rows: -2
Enter number of columns: -3
Rows and columns must be positive. Enter again.
Enter number of rows: 3
Enter number of columns: 2
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[2][0] = 3
a[0][1] =
4
a[1][1] = 5
a[2][1] = 6
Array elements in row-major order:
1 4
2 5
3 6
```

Question 2 - Compilation

```
(kali㉿kali)-[~/Desktop/apple]
$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~/Desktop/apple]
$
```

Question 2 - Test Case 1 (2 x 3)

```
(kali@kali)-[~/Desktop/apple]  
$ ./hdoc28
```

Enter number of rows and columns: 2 3

Enter elements of the 2D array:

1 2 3 4 5 6

2D array:

1 2 3

4 5 6

1D array in row-major order:

1 2 3 4 5 6

Question 2 - Test Case 2 (3 x 2)

```
(kali@kali)-[~/Desktop/apple]  
$ ./hdoc28
```

Enter number of rows and columns: 3

2

Enter elements of the 2D array:

1

2

3

4

5

6

2D array:

1 2

3 4

5 6

1D array in row-major order:

1 2 3 4 5 6